Optimizing Recovery Logic in Speculative High-Level Synthesis

Dylan Leothaud*, Jean-Michel Gorius*, Simon Rokicki*, Steven Derrien†
*Univ Rennes, Inria, CNRS, IRISA
†UBO, Lab-STICC

2025 62nd ACM/IEEE Design Automation Conference (DAC) | 979-8-3315-0304-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/DAC63849.2025.11133015

***Abstract*—High-Level Synthesis (HLS) excels at handling compute-intensive loops with straightforward control but struggles to identify parallelism in kernels with complex and irregular control-flow. To address this, novel scheduling techniques based on speculation have been introduced. While these methods outperform traditional static scheduling, they also introduce significant area overhead, particularly in the rollback control logic. Optimizing the cost of this rollback control logic remains an open challenge. In this work, we show how it is possible to simplify and/or eliminate rollback logic using a combination of static analysis and linear programming. Our results show improvements in both execution throughput and area cost.**

I. Introduction

High-Level Synthesis (HLS) has emerged as a powerful technique for designing hardware accelerators, with the potential to rival manual designs. However, current HLS tools still struggle with complex data-dependent control flow and memory accesses, largely due to the use of static scheduling techniques that cannot utilize runtime information.

There has been growing research interest in extending HLS tools to support dynamic [1]–[8] and speculative [5], [8]–[11] execution mechanisms to address the shortcomings of static scheduling. While dynamic scheduling has been studied extensively, speculative scheduling has received comparatively less attention, partly because of the significant area and clock speed overheads introduced by the speculative circuits generated by corresponding toolchains, which often outweigh the potential performance benefits. Reducing overhead in speculative circuits is a fundamental issue that needs to be addressed to make speculative execution techniques practical for HLS. The recovery logic, which tracks program states and restores the last valid state in the event of a misspeculation, is the primary source of this overhead.

This work shows that current approaches result in unnecessarily complex recovery logic. We address this issue by proposing a set of techniques to optimize the rollback logic and its placement within the datapath. Specifically, our contributions are as follows:

- We introduce two transformations to simplify or eliminate unecessary checkpoint/recovery logic.
- We formulate and solve the problem of minimum cost checkpointing.
- We validate our approach experimentally, showing an average throughput improvement of 23%, and an average resource utilization reduction of 10%.

This paper is organized as follows. Section II provides background information on speculative scheduling. Section III presents our approach, Section IV addresses its experimental validation, and discusses related work. Section VI concludes the paper.

II. Background

This section recalls key concepts relevant to HLS scheduling, focusing on speculative pipelining techniques and their associated recovery mechanisms.

A. Scheduling in High-Level Synthesis

Custom hardware relies on parallelism and customization to achieve high performance. From a compiler's perspective, *identifying* and *exploiting* parallelism is challenging. Notably, *loop pipelining* [12] has proven to be a critical transformation in HLS due to its ability to automatically generate deep and wide pipelined datapaths, even in the presence of loop-carried dependencies. However, some kernels cannot benefit from loop pipelining, often because of unpredictable control flow or memory access patterns. Speculative (and dynamic) loop pipelining (SLP) techniques enable more efficient implementations by anticipating the outcome of conditionals and memory accesses.

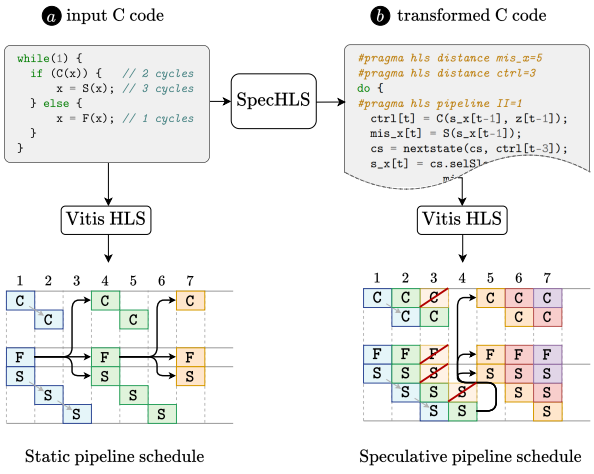


Fig. 1. Speculative pipelining as a source-level transformation in SpecHLS.

The SpecHLS flow enables SLP by extending existing HLS toolchains through source-level transformations to support dynamic and speculation execution [9], [11]. Figure 1 illustrates this approach on a simple loop where the execution path of an

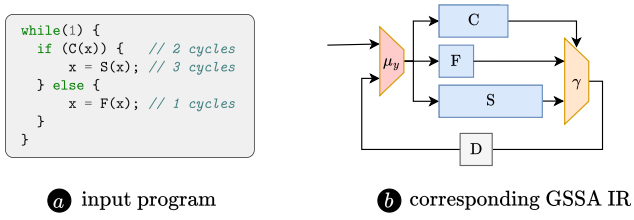
iteration depends on the outcome of a data-dependent decision. In such cases, SOTA HLS tools will assume the worst-case scenario and derive a schedule with an Initiation Interval (II) of $\mathrm{II} = 3$, as illustrated in part (a). In contrast, SpecHLS produces optimized C++ code that takes advantage of speculation by considering that the `F` path is always taken, leading to a design with $\mathrm{II} = 1$. When a mispeculation occurs, the loop cancels its pending operations and restarts from a valid state.

B. The SpecHLS flow and its compiler IR

The SpecHLS flow uses a variant of the Gated-SSA representation (GSSA) [13], [14] as its core program Intermediate Representation (IR). In GSSA, the φ-nodes of traditional SSA form [15] are replaced with *gating nodes*. The type of gating node that replaces a given φ-node depends on the context in which the φ-node appears. Gated-SSA enables us to work with fully-predicated φ-nodes, represented as μ- and γ-nodes. The following gating nodes are particularly relevant to our discussion:

- μ-nodes are placed in loop headers. They take three arguments, $\mu(p, i_x, l_x)$, where p is the loop exit condition, i_x is the initial value of variable x, and l_x is the value of x after a loop iteration.
- γ-nodes are placed at join points in the control-flow graph, such as the end of conditional structures. They encode a predicate, determining which value to select when execution reaches them. We denote them by $\gamma(c, x_0, x_1, \ldots, x_n)$, where c is the predicate, and $x_0, x_1, \ldots, x_n$ are the possible values for variable x.
- delay nodes (δ) capture inter-iteration producer/consumer relations. For example, data reuse spanning two iterations will be represented as a 2δ node.

Figure 2 illustrates the GSSA operators utilized in this work. For clarity, our examples omit the loop exit condition from μ-nodes.



(a) input program (b) corresponding GSSA IR

Fig. 2. Extended Gated-SSA operators. μ-nodes represent loop headers, and γ-nodes encode control-flow decisions. Inter-iteration dataflow is modeled using δ-nodes.

C. Speculation as an IR transformation

Because they capture conditionals, γ-nodes expose speculation opportunities. For instance, if we target $T_{clk} = 10\,\mathrm{ns}$ and assume the existence of pipelined implementations for the operators F, C, and S, with latencies of 1, 2, and 3 cycles respectively, speculating that the γ-node in Figure 2 (b) selects the result from the F operation allows pipelining with an $\mathrm{II} = 1$, leading to the speculative pipelined schedule depicted in part (b) of Figure 1.

To be considered for speculation, a given γ-node must (i) expose a long delay over at least one of its data or control inputs, (ii) expose a short delay over one of its input data ports, and (iii) the probability for the latter path to be taken must be higher than a given threshold. Determining which γ-nodes to speculate is a complex problem, which has been addressed in prior work [8].

The key idea behind the SpecHLS transformation flow is to expose the pipeline depth directly in the IR by extending the reuse distance through explicit delays (δ nodes), weave the appropriate hazard recovery logic, and then let the HLS back-end tool map each loop kernel operations to a specific pipeline stage. More precisely, the transformation includes two key steps:

1) Delays (δ-nodes) are inserted on non-speculated paths (e.g., path involving `S` and `C`) as depicted in Figure 3 (b). Those delays increase the reuse distance for loop-carried dependencies and allow the back-end HLS toolchain scheduler to obtain an initiation interval of $\mathrm{II} = 1$.
2) Recovery logic is also inserted to handle mispeculations. This logic consists of (i) rollback nodes (R nodes), with an internal buffer of a given *depth*, for restoring past inputs, and a Finite State Machine (FSM) depicted in Figure 3 (b).

The transformation is illustrated in part (b) of Figure 3, where two extra delays have been added along the slow path, one along the conditional path, and a rollback node of depth three is used to store the last three values for variable `x`. The ways the HLS back-end uses these additional nodes is illustrated in part (c) of Figure 3, where the rollback node has been lowered into a combination of a shift register and a multiplexer, and where the static pipeline scheduler has moved delays to enable full pipelining.

The SpecHLS flow supports multiple concurrent speculations [11]. The flow also handles speculative memory accesses, which do not require rollback nodes (they use some form of LSQ instead [16]). Detailing the whole flow is out of the scope of the present work, and the reader is invited to refer to previous work on the topic [9], [11]. Since this work is focused on recovery optimizations, we describe where and how rollback nodes are currently inserted in the SpecHLS flow:

- Rollback nodes are inserted at the output of all μ nodes in the Strongly Connected Component (SCC) of the GSSA graph containing the speculated gammas.
- In the case of multiple speculations (*i.e.*, multiple γ-nodes within the same SCC), rollback nodes are inserted at the output of all speculated γ-nodes.
- The recovery depth of the rollback node is obtained by determining the longest path (in terms of clock cycles) in the datapath for a mispeculation to be detected (2 cycles in our example)

D. Problem statement

A significant fraction of the area overhead caused by SLP is linked to the mispeculation recovery process, which needs

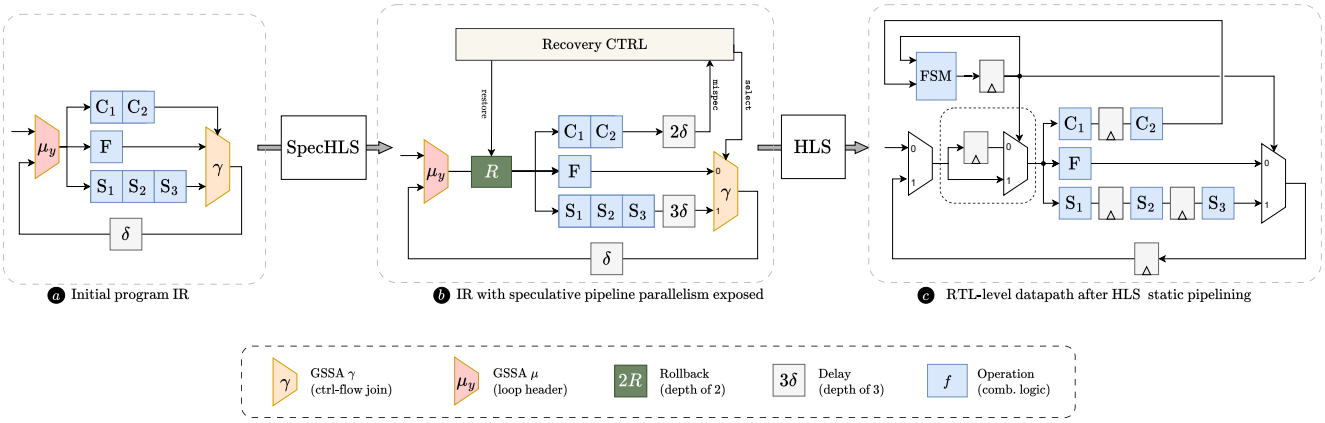


Fig. 3. Overview of the program IR transformations involved in the SpecHLS compilation flow.

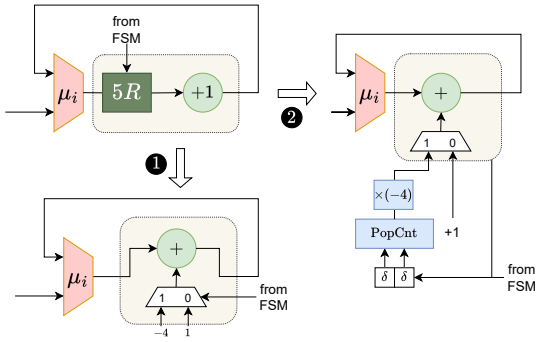


Fig. 4. Rollback simplification for a loop increment operation.

to keep track of previous loop iteration states (through multiplexers and shift registers) and manage the recovery sequence using dedicated control signals from the FSM. In addition, rollback nodes also negatively impact clock speed as they end up being part of the new IR critical path. The problem we address in this work is minimizing this overhead through a set of transformations, which we describe in the following section.

III. Proposed Approach

In practice, rollback nodes can be placed at different locations without impacting correctness, and in many cases, this logic can be optimized. In the following, we describe these optimizations and show how we can formulate a global optimization problem using a simple Linear Programming model.

A. Rollback simplification

Sometimes, a variable can be reverted to a previous state without saving its prior values. This optimization applies to *reversible* operations, *i.e.*, computations where the original input can be reconstructed from the output. In other words, given the result of the operation, it is possible to reverse the process and precisely recover the input. Increment operations follow this pattern: their rollback can be performed using a simple subtraction, where the value to be subtracted is either a constant or can be computed with minimal resource overhead.

Consider, for example, a kernel snippet that increments a variable `i` by `1` during each iteration, along with its GSSA intermediate representation, as shown in Figure 4. Two scenarios need to be considered depending on whether multiple speculations are involved.

- In the case of a single speculation ❶, the datapath will either increment `i` by `1` or recover from a speculation through a rollback. A rollback with a depth d simply requires decrementing the variable by $1 \times d$. This subtraction operator can then be merged with the subsequent adder to produce the simplified circuit shown as the result transformation ❶.
- The situation becomes more complex with multiple speculations ❷, since several speculative execution states may overlap. In such cases, tracking the number of pending rollbacks within the speculation window is necessary to determine the value to subtract. This tracking can be effectively implemented using a single-bit-wide shift register whose depth corresponds to the rollback maximum depth (to keep pending rollback requests). This shift register is paired with a pop-count structure, as depicted in transformation ❷, whose result is used to determine the actual iterator value. This structure can be easily generalized to support rollbacks with multiple depths at a slight increase in area cost.

B. Rollback elimination

Rollbacks revert execution to previous iterations through checkpointing, ensuring the datapath can restart from a valid state after recovering from misspeculation. In the SpecHLS flow, rollbacks are inserted at the output of every μ-nodes and, in cases of multiple speculations, at the output of certain γ-nodes. While this method guarantees correctness, it can be overly conservative, as the output of the rollback node may not always be necessary.

This scenario is demonstrated in part ⓐ of Figure 5, where a rollback is inserted after the μ-nodes. This rollback is associated with the misspeculation recovery process from the γ-node. In this example, the reader will notice that the output of the rollback does *never* contribute to the actual result because it will always be overshadowed by the output of the `F` function (if no misspeculation occurred) or the `S` functions (when recovering

from a mispeculation). This rollback node can, therefore, safely be removed without affecting the program's correctness.

Such a simplification is, however, not always possible, as demonstrated in part (b) of Figure 5, where the output of the μ-node is forwarded beyond the γ-node through the `H` operation. This rollback node is mandatory in such a situation since it will contribute to the next value of x during the recovery.

More formally, the condition for removing a rollback node is based on postdominance properties in the graph. Postdominance can be defined as follows: a node d in a graph is said to *postdominate* another node n if every path to the exit node that starts at n also has to go through d. In our context, a rollback node can be eliminated when it is post-dominated by a speculative γ-node.

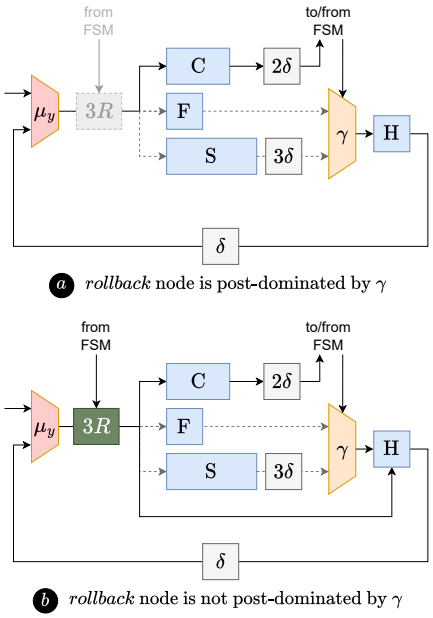


Fig. 5. Rollback elimination based on post-domination.

The issue becomes more complex when dealing with multiple speculated γ-nodes, as a rollback node, can only be eliminated if it is post-dominated by all of the speculated γ-nodes, which is rarely the case. However, our implementation addresses several specific scenarios to enhance the effectiveness of the analysis. For instance, when multiple speculated γ-nodes share the same speculated control signal, they are treated as a single node for the post-domination analysis.

C. Rollback placement

As discussed in Section II, the SpecHLS flow places rollback nodes in the IR graph at specific points (after μ-and γ-nodes). However, this placement is somewhat arbitrary, as alternative placement options can ensure correctness while potentially reducing area overheads. For example, rollback nodes can be moved forward in the IR graph, as illustrated in Figure 6. To be valid, the move must enforce certain conditions:

- A rollback node r may only be moved past another node s in the graph if all predecessors of r are also rollback nodes. Figure 6 depicts such a transformation. This rule ensures consistency in the information flow within the IR graph. This transformation strongly resembles the well-known register retiming optimization [17].
- Rollback moves must not cross inter-iteration boundaries to avoid de-synchronizing the control signal from the FSM, which could otherwise arrive one or more cycles too late. Consequently, we also prohibit rollback moves across delay and μ-nodes, which mark iteration boundaries.

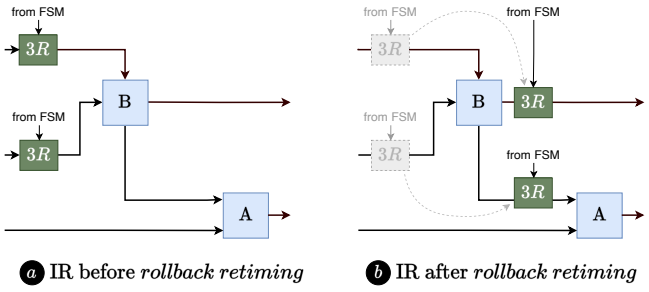


Fig. 6. Example of *forward* retiming of a *rollback* node.

As discussed previously, simplifying or eliminating rollback nodes depends on their position within the graph. Since these nodes can be moved using the transformations described above, we aim to derive a set of elementary moves that achieve the optimal placement of all rollback nodes in the graph.

This optimization is illustrated in Figure 7. In this example, two variables (`x` and `y`) are updated at every iteration (as shown in (a)). Assuming that a speculation decision (of depth 3) is applied to the γ-node, two rollback nodes (with a cost of 3 registers each) must be inserted at the μ nodes (as shown in (b)), resulting in a cost of $2 \times 3 = 6$ registers. It is, however, possible to move those rollback nodes past `B` and `A` nodes to obtain an optimized IR with a cost of 3 registers, since two of them can be eliminated (they become post-dominated by the γ-node), as illustrated in (c). In the following, we show how it is possible to derive such an optimal set of moves automatically.

D. Rollback minimization as a retiming problem

As mentioned previously, the rollback move operation can be seen as a form of retiming [17], a technique used in digital circuit optimization where the positions of registers within a circuit are adjusted to optimize both the number of registers and the clock period. Our rollback placement problem is very similar to the registers cost minimization problem, which we recall below.

Consider a synchronous digital circuit represented by a directed graph $G = (V, E)$, where:

- V is the set of vertices representing the computational nodes in our program representation.
- E is the set of directed connection edges between these blocks, where each edge $(u, v) \in E$ has a weight $w(u, v)$ indicating the number of delays on that edge, and a cost $c(u, v)$ which accounts for the edge data's bit-width.

Retiming involves shifting registers from one edge to another without changing the logical behavior of the circuit. Let

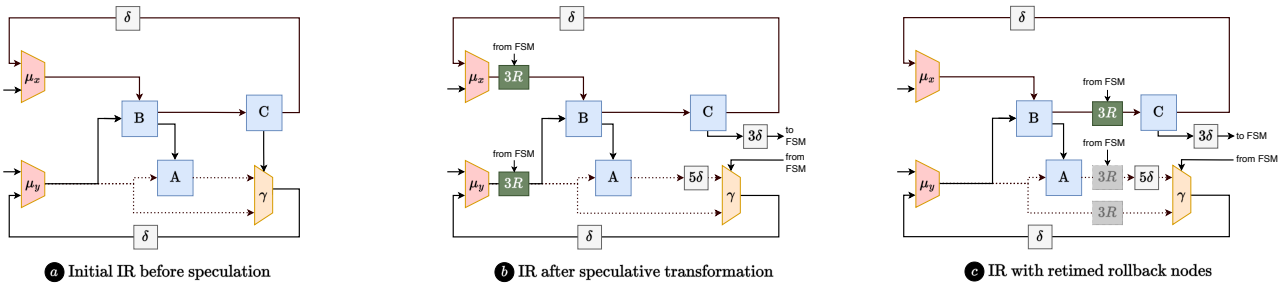


Fig. 7. Example of rollback optimization.

$r : V \to \mathbb{Z}$ be a retiming function, where $r(v)$ represents the retiming value (i.e., the number of registers added to or removed from the edges incident to vertex v).

The register minimization problem can be modeled using a Linear Programming formulation, which can be solved in polynomial time [17]. The objective is to minimize the cost of registers in the circuit after retiming. The LP problem is as follows:

$$\text{Minimize} \quad \sum_{(u,v)\in E} c(u,v) \times (w(u,v) + r(v) - r(u))$$

After retiming, we must also ensure that the number of registers on any edge (u, v) must be non-negative:

$$r(v) - r(u) \leq w(u,v) \quad \forall (u,v) \in E$$

We can reuse this formulation as is for our rollback optimization problem, but with the following modifications.

- we model a rollback node of depth d as a d-deep retiming register. We also prevent retiming over delay nodes and force $r(v) = 0$ for such nodes.
- for edges whose successors are post-dominated by a speculated gamma node (and for which rollback nodes can be eliminated), we set $c(u, v) = 0$.

IV. Experimental validation

This Section evaluates the relevance of our approach from a qualitative and quantitative perspective. We aim to demonstrate that our transformations reduce area costs and improve clock speed by reducing the designs' critical path length. We do not, however, explore the trade-offs implied by the use of speculation in HLS designs, since those orthogonal issues have been discussed in prior works [9]–[11].

A. Experimental Setup

Our technique is implemented within the SpecHLS source-to-source compiler [11], which is based on the GeCoS framework [18], coupled to the `lpSolve` solver [19]. We use Vitis HLS 2022.2 with an XC7A25 FPGA as the target device, with a target clock period of 10 ns. Existing HLS benchmarks [20], [21] primarily focus on kernels with regular control-flow and memory access patterns, favoring applications that work well with state-of-the-art HLS tools. These benchmarks are not suitable for evaluating our approach. Instead, we selected a set of benchmarks that exhibit data-dependent control flow and memory accesses, where speculation can be applied. These benchmarks were collected from previously published work [1], [11], [22]–[24].

We compare resource usage and performance for each benchmark across four implementations: a baseline implementation (based solely on Vitis HLS), shown in gray, and three SpecHLS versions. The first SpecHLS version (in green) does not incorporate our proposed rollback optimization strategies, the second one (in blue) includes our rollback optimization strategy, and the third (in orange) is a speculative version where we assume no mispeculation occurs, allowing the removal of all recovery logic. We use this last version to represent the hypothetical maximum reduction in recovery logic area cost.[1]

B. Experimental results

Resource usage after RTL synthesis, and for a target clock of $T_{\text{clk}} = 10$ ns is shown in Figure 8 and is reported in terms of LUTs, FFs, SRLs, BRAMs, and DSPs for each benchmark. Performance, shown in Figure 8, is measured in terms of absolute running time. The baseline's running time is computed as $T_{\text{exec}} = \text{II} \times T_{\text{clk}}$. In contrast, for the speculative versions, it is calculated as $T_{\text{exec}} = \text{CPI} \times T_{\text{clk}}$, where CPI (Cycles Per Iteration) accounts for mispeculation penalties. For benchmarks where the mispeculation rate depends on the dataset, results may vary depending on the mispeculation rate. We illustrate this phenomenon on the `hist` benchmark at the bottom of Figure 8, where we vary the mispeculation rate (2%, 10%, 30%) following the approach suggested by Derrien *et al.* [9] to reflect the impact of these datasets on performance.

Results show that our approach improves area and performance for all but one benchmark, with an average throughput improvement of 13% and a reduction in FF+LUT+SRL area of 10%. There is, however, one outlier: for the `nr` benchmark, our analysis failed to identify any rollback optimization opportunities. Upon conducting a manual analysis, we found a rollback node that could have been eliminated, but detecting this would require more complex analyses (such as model checking or SMT solvers), which are beyond the scope of this work.

We note that speculative loop pipelining is a *last mile* optimization, which implies that is intended to extract parallelism from applications that are inherently hard to optimize for HLS

[1]The area overhead, when compared to the golden version, is due to the fact that designs with a unit initiation interval (II = 1) use more resources than those with II > 1, as full pipelining prevents resource reuse.

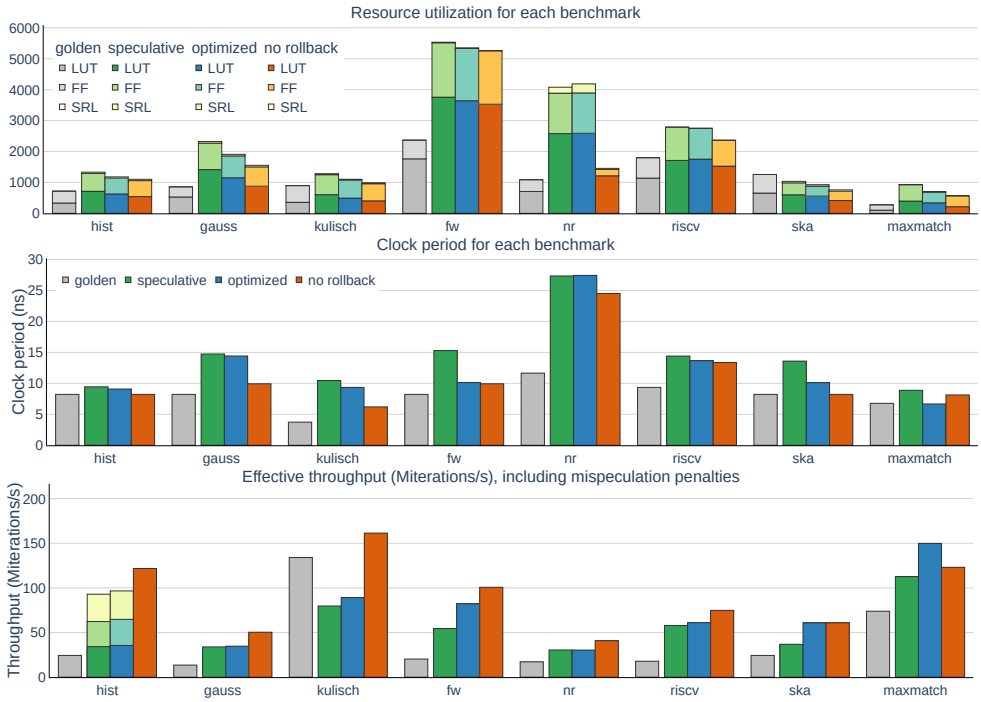


Fig. 8. Resource utilization, clock period, and effective throughput for our set of benchmarks on an XC7A25 FPGA device. The throughput graph for the speculative versions of `hist` shows the effective throughput for a mispeculation rate of 2%, 10%, and 30%, using overlaid bars, from highest to lowest. The version without rollbacks assumes a mispeculation rate of 0%.

toolchains. Consequently, the benefits one can obtain from using speculative HLS techniques often outweigh the potential scalability concerns that other more systematic compiler optimizations (such as loop transformations) may face in terms of usability. Evaluation of our approach on different boards and larger designs is left as future work.

V. Related Work and Discussion

This section covers three key points: (i) how our approach differs from existing work on speculative and dynamic HLS, (ii) the distinctions between our rollback optimization technique and compiler optimizations for checkpointing and recovery in Thread-Level Speculation for multi-core systems, and (iii) how our objectives differ from checkpointing mechanisms in large-scale HPC systems.

Several works have tackled the challenge of leveraging speculation in HLS tools, with speculative dataflow circuits [10] standing out as one of the most advanced approaches. The latter introduce speculative execution capabilities into elastic hardware designs. Speculative data tokens, generated by *speculators*, propagate through a network of elastic components synchronized via handshake signals. While this token-based design is effective, it is less general than SpecHLS's approach, since it cannot handle multiple intertwined speculations [11]. Furthermore, the distributed nature of speculative dataflow circuits limits rollback optimization opportunities by restricting recovery to computational replay.

There has also been extensive research on speculative parallelization, mainly focusing on recovery mechanisms and enhancing the efficiency of speculative execution in multi-threaded environments for multi-core processors [25]. For example Tian *et al.* [26] propose a speculative parallelization framework incorporating incremental recovery, which reduces the overhead from speculative failures by rolling back only the minimal necessary computations. Yagüez *et al.* [27] explore different squashing alternatives in software-based speculative parallelization, evaluating strategies for handling mispeculations and assessing their impact on system performance. However, these approaches operate at a much coarser granularity than the iteration level, making them unsuitable for our context.

Checkpointing and recovery mechanisms are also commonly used to address hardware failures in HPC or intermittent systems, with several works proposing compiler support for such mechanisms [28]–[30]. However, these approaches typically operate at the software level (with some exceptions [31]), work at a very coarse granularity, or assume that such failures are rare events, making them significantly different from the fine-grained, frequent speculation context we address.

VI. Conclusion

Speculative HLS opens up interesting new design perspectives, as it allows for the acceleration of complex, control-dominated kernels. It does, however, come at a significant area cost. In this paper, we show how to reduce the area overhead of the rollback logic in speculatively scheduled hardware designs. Through a combination of static analysis and linear programming, we are able to synthesize smaller designs while simultaneously improving the throughput of the synthesized hardware by 13% on average, with some benchmarks exhibiting more than 40% performance improvement.

ACKNOWLEDGMENT

This work is partly funded by the French National Research Agency (ANR), as part of the LOTR project[2] (ANR-23-CE25-0016)

REFERENCES

[1] M. Alle, A. Morvan, and S. Derrien, "Runtime Dependency Analysis for Loop Pipelining in High-Level Synthesis," in *Proceedings of the 50th Annual Design Automation Conference*, ser. DAC '13. New York, NY, USA: Association for Computing Machinery, 2013. [Online]. Available: https://doi.org/10.1145/2463209.2488796

[2] J. Cong, J. Lau, G. Liu, S. Neuendorffer, P. Pan, K. Vissers, and Z. Zhang, "FPGA HLS Today: Successes, Challenges, and Opportunities," *ACM Transactions on Reconfigurable Technology and Systems*, vol. 15, no. 4, pp. 51:1–51:42, Aug. 2022. [Online]. Available: https://dl.acm.org/doi/10.1145/3530775

[3] S. Dai, R. Zhao, G. Liu, S. Srinath, U. Gupta, C. Batten, and Z. Zhang, "Dynamic hazard resolution for pipelining irregular loops in high-level synthesis," in *Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, ser. FPGA '17. New York, NY, USA: Association for Computing Machinery, 2017, p. 189–194. [Online]. Available: https://doi.org/10.1145/3020078.3021754

[4] L. Josipović, R. Ghosal, and P. Ienne, "Dynamically Scheduled High-level Synthesis," in *Proceedings of the 2018 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, ser. FPGA '18. New York, NY, USA: Association for Computing Machinery, Feb. 2018, pp. 127–136. [Online]. Available: https://dl.acm.org/doi/10.1145/3174243.3174264

[5] V. Lapotre, P. Coussy, C. Chavet, H. Wouafo, and R. Danilo, "Dynamic branch prediction for high-level synthesis," in *2013 23rd International Conference on Field programmable Logic and Applications*, 2013, pp. 1–6.

[6] J. Cheng, J. Wickerson, and G. A. Constantinides, "Probabilistic Scheduling in High-Level Synthesis," in *2021 IEEE 29th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*, 2021, pp. 195–203.

[7] R. Szafarczyk, S. W. Nabi, and W. Vanderbauwhede, "Compiler Discovered Dynamic Scheduling of Irregular Code in High-Level Synthesis," in *2023 33rd International Conference on Field-Programmable Logic and Applications (FPL)*. Los Alamitos, CA, USA: IEEE Computer Society, sep 2023, pp. 1–9. [Online]. Available: https://doi.ieeecomputersociety.org/10.1109/FPL60245.2023.00009

[8] D. Leothaud, J.-M. Gorius, S. Rokicki, and S. Derrien, "Efficient design space exploration for dynamic & speculative high-level synthesis," in *34th International Conference on Field-Programmable Logic and Applications*, 2024.

[9] S. Derrien, T. Marty, S. Rokicki, and T. Yuki, "Toward Speculative Loop Pipelining for High-Level Synthesis," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 39, no. 11, pp. 4229–4239, Nov. 2020.

[10] L. Josipović, A. Guerrieri, and P. Ienne, "Speculative Dataflow Circuits," in *Proceedings of the 2019 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, ser. FPGA '19. New York, NY, USA: Association for Computing Machinery, 2019, p. 162–171.

[11] J.-M. Gorius, S. Rokicki, and S. Derrien, "SpecHLS: Speculative Accelerator Design Using High-Level Synthesis," *IEEE Micro*, vol. 42, no. 5, pp. 99–107, 2022.

[12] M. Lam, "Software Pipelining: An Effective Scheduling Technique for VLIW Machines," in *Proceedings of the ACM SIGPLAN 1988 Conference on Programming Language Design and Implementation*, ser. PLDI '88. New York, NY, USA: Association for Computing Machinery, 1988, p. 318–328. [Online]. Available: https://doi.org/10.1145/53990.54022

[13] P. Tu and D. Padua, "Efficient building and placing of gating functions," in *Proceedings of the ACM SIGPLAN 1995 Conference on Programming Language Design and Implementation*, ser. PLDI '95. New York, NY, USA: Association for Computing Machinery, 1995, p. 47–55.

[14] ——, "Gated SSA-Based Demand-Driven Symbolic Analysis for Parallelizing Compilers," in *Proceedings of the 9th International Conference on Supercomputing*, ser. ICS '95. New York, NY, USA: Association for Computing Machinery, 1995, p. 414–423. [Online]. Available: https://doi.org/10.1145/224538.224648

[15] R. Cytron, J. Ferrante, B. K. Rosen, M. N. Wegman, and F. K. Zadeck, "Efficiently computing static single assignment form and the control dependence graph," *ACM Transactions on Programming Languages and Systems (TOPLAS)*, vol. 13, no. 4, pp. 451–490, 1991.

[16] J.-M. Gorius, S. Rokicki, and S. Derrien, "A Unified Memory Dependency Framework for Speculative High-Level Synthesis," in *Proceedings of the 33rd ACM SIGPLAN International Conference on Compiler Construction*, ser. CC 2024. New York, NY, USA: Association for Computing Machinery, 2024, p. 13–25. [Online]. Available: https://doi.org/10.1145/3640537.3641581

[17] C. E. Leiserson and J. B. Saxe, "Retiming synchronous circuitry," *Algorithmica*, vol. 6, no. 1, pp. 5–35, 1991.

[18] A. Floc'h, T. Yuki, A. El-Moussawi, A. Morvan, K. Martin, M. Naullet, M. Alle, L. L'Hours, N. Simon, S. Derrien, F. Charot, C. Wolinski, and O. Sentieys, "GeCoS: A framework for prototyping custom hardware design flows," in *2013 IEEE 13th International Working Conference on Source Code Analysis and Manipulation (SCAM)*, 2013, pp. 100–105.

[19] M. Berkelaar *et al.*, "Package 'lpsolve'," 2015.

[20] Y. Hara, H. Tomiyama, S. Honda, H. Takada, and K. Ishii, "CHStone: A benchmark program suite for practical C-based high-level synthesis," in *2008 IEEE International Symposium on Circuits and Systems (ISCAS)*, 2008, pp. 1192–1195.

[21] Y. Zhou, U. Gupta, S. Dai, R. Zhao, N. Srivastava, H. Jin, J. Featherston, Y.-H. Lai, G. Liu, G. A. Velasquez, W. Wang, and Z. Zhang, "Rosetta: A Realistic High-Level Synthesis Benchmark Suite for Software Programmable FPGAs," in *Proceedings of the 2018 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, ser. FPGA '18. New York, NY, USA: Association for Computing Machinery, 2018, p. 269–278. [Online]. Available: https://doi.org/10.1145/3174243.3174255

[22] J. Cheng, L. Josipović, G. A. Constantinides, P. Ienne, and J. Wickerson, "DASS: Combining Dynamic & Static Scheduling in High-Level Synthesis," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 41, no. 3, pp. 628–641, 2022.

[23] J. Liu, J. Wickerson, S. Bayliss, and G. A. Constantinides, "Polyhedral-Based Dynamic Loop Pipelining for High-Level Synthesis," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 37, no. 9, pp. 1802–1815, Sep. 2018.

[24] J.-M. Gorius, S. Rokicki, and S. Derrien, "Design Exploration of RISC-V Soft-Cores through Speculative High-Level Synthesis," in *2022 International Conference on Field-Programmable Technology (ICFPT)*, 2022, pp. 1–6.

[25] N. Vachharajani, R. Rangan, E. Raman, M. J. Bridges, G. Ottoni, and D. I. August, "Speculative Decoupled Software Pipelining," in *16th International Conference on Parallel Architecture and Compilation Techniques (PACT 2007)*, 2007, pp. 49–59.

[26] C. Tian, C. Lin, M. Feng, and R. Gupta, "Enhanced speculative parallelization via incremental recovery," *ACM SIGPLAN Notices*, vol. 46, no. 8, pp. 189–200, 2011.

[27] Á. G. Yágüez, D. R. Llanos, and A. Gonzalez-Escribano, "Squashing alternatives for software-based speculative parallelization," *IEEE Transactions on Computers*, vol. 63, no. 7, pp. 1826–1839, 2013.

[28] B. Yarahmadi and E. Rohou, "Compiler optimizations for safe insertion of checkpoints in intermittently powered systems," in *Embedded Computer Systems: Architectures, Modeling, and Simulation: 20th International Conference, SAMOS 2020, Samos, Greece, July 5–9, 2020, Proceedings 20*. Springer, 2020, pp. 169–185.

[29] S.-E. Choi and S. J. Deitz, "Compiler support for automatic checkpointing," in *Proceedings 16th Annual International Symposium on High Performance Computing Systems and Applications*. IEEE, 2002, pp. 213–220.

[30] X. Yang, P. Wang, H. Fu, Y. Du, Z. Wang, and J. Jia, "Compiler-assisted application-level checkpointing for mpi programs," in *2008 The 28th International Conference on Distributed Computing Systems*. IEEE, 2008, pp. 251–259.

[31] A. Bourge, O. Muller, and F. Rousseau, "Automatic high-level hardware checkpoint selection for reconfigurable systems," in *2015 IEEE 23rd Annual International Symposium on Field-Programmable Custom Computing Machines*. IEEE, 2015, pp. 155–158.

[2] https://lotr.irisa.fr/